Gap Stack + Cost of Delay — Design Doc

Nov 7, 2026 · @Harsh

Summary

We are building one web tool that shows a nonprofit the funding gap on each affordable home, how much of that gap comes from waiting on the city, and who can pay for the rest. It covers Lane 2 (Clear it) and Lane 3 (Fund it).

The problem. An affordable home in Orange County costs more to build than the family can pay. In the organizers' synthetic example, a home costs $535k and the family can pay $285k (both labeled ASSUMPTION in the repo). Part of that cost is time: loan interest, taxes and insurance paid while approvals drag on. Today a finance lead pieces this together by hand from PDFs.

What we build. Enter a project (city, homes, cost per home, income mix). The tool returns:

1. What each family can legally pay, from state law, the 2026 income limits and today's mortgage rate
2. The gap per home
3. The share of the gap caused by time to permit in that city, from state progress-report data
4. Which programs can fill the gap, each marked confirmed, funds unconfirmed or unknown, and what is left to fundraise

Every number shows its source. Every guess is labeled ASSUMPTION.

One-line pitch: "Our tool shows a nonprofit how big the funding gap is on each home, how much of it comes from waiting on the city, and who can help pay for the rest."

User

Our one user is the finance lead at a small Orange County nonprofit that builds for-sale homes for families around 80% of area median income. This is a role, not a real person (the rules forbid personal information).

	Before	After
Time to size one project's gap	About two weeks of spreadsheets and calls (ASSUMPTION, confirm with a practitioner)	About 10 minutes

	Before	After
Income limits	Looked up in PDFs, may be stale	Right table picked, revision date shown
City timelines	A guess	Months to permit from state data, with project count
Funding programs	Phone calls to see who has money	Listed with status: confirmed, funds unconfirmed, unknown
Board pitch	Gap total with no breakdown	Gap split into land, build, time; one-page funder memo

Also helped, but not built for: city staff (see what their timelines cost) and donors (see what a gift pays for, shown by the finance lead).

Action: before lunch, ask a practitioner in the lane room how long sizing a gap takes today. Replace the ASSUMPTION above with their answer.

How it works

Every home must balance the organizers' equation. The tool computes each term in code; Claude never does the arithmetic.

```
\text{family price} + \text{subsidies} + \text{fundraising} - \text{cost to create} = 0
```

1. Affordable price (Lane 3). Health and Safety Code 50052.5 caps owner housing cost as a share of an income band. Confirm the exact percentages, the household-size rule and the pre/post-1991 branch on the statute page before coding them. The repo says the full tables were not read.

```
M = \frac{\text{share} \times \text{income limit}(\text{band}, \text{household size})}{12} - \text{tax} - \text{insurance} - \text{HOA} - \text{utilities}
```

```
\text{loan} = M \times \frac{1 - (1 + r)^{-360}}{r}, \quad r = \frac{\text{annual rate}}{12}
```

Family price = loan + down payment. Tax, insurance, HOA, utilities and down payment are ASSUMPTION inputs, shown on screen.

2. Cost to create. Land, construction, fees, plus the time cost from step 3.

3. Time cost (Lane 2).

```
\text{time cost per home} = \text{months to permit (city)} \times
\text{monthly carrying cost per home}
```

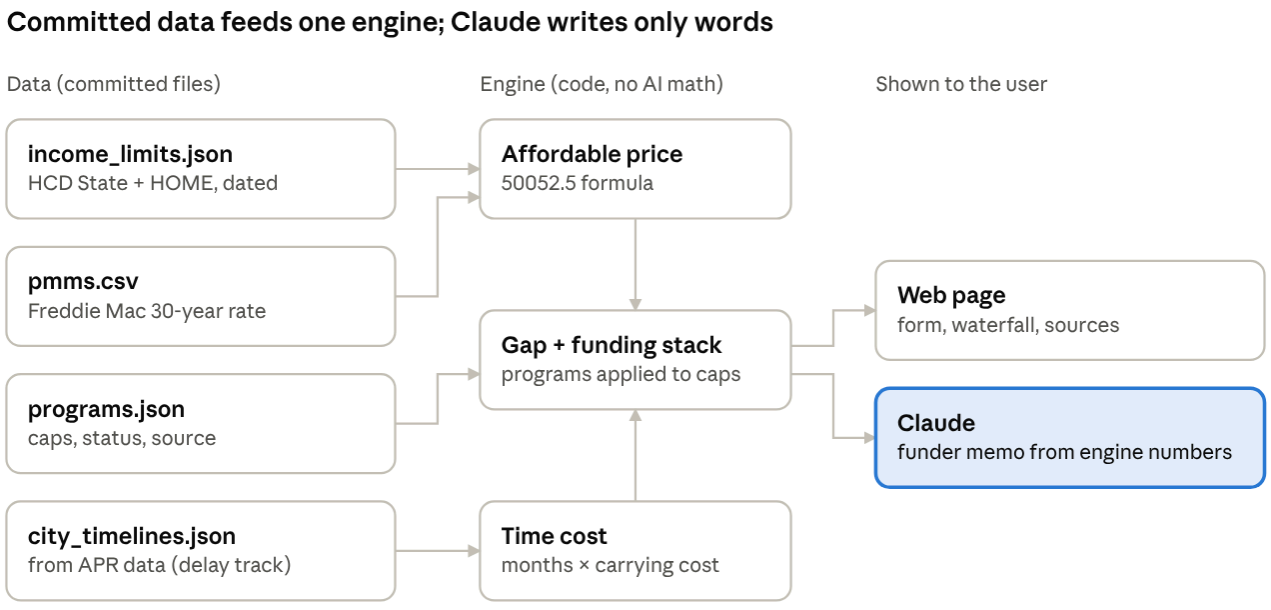
Months come from the state progress-report data (or the fallback). Monthly carrying cost is an ASSUMPTION input: loan interest, property tax, insurance, management.

4. Gap and funding stack. Gap = cost to create − family price. Apply each eligible program up to its cap, in order. What remains is the fundraising need. The chart shows the gap shrinking to $0.

Check: feed in the synthetic deal (`lanes/3-fund-it/data/synthetic-example-deal.json`) with its family price fixed at $285k, $100k subsidies and $55k rent earned while waiting. The tool must return the $95k fundraising gap.

Architecture

Four committed data files feed one calculation engine; the page and Claude only display its results. Stack suggestion: Python + Streamlit (fastest to a working page), or whatever the builders know best.



architecture · 4 data files, 3 engine steps, 2 outputs

The delay track owns only `city_timelines.json` ; everything else reads it. Stretch: Claude reads a program PDF and drafts a new `programs.json` row for a person to approve.

File layout

```
data/
  income_limits.json    # HCD State + HOME tables, with revision dates
  pmms.csv              # Freddie Mac rate history
  programs.json         # one row per funding program
  city_timelines.json   # delay track output
  synthetic_deal.json   # test case only
engine/
  affordability.py      # family price from 50052.5
  time_cost.py          # months x carrying cost
  stack.py              # gap, programs, fundraising left
delay/
  build_timelines.py    # APR rows -> city_timelines.json
app.py                  # the page
memo.py                 # Claude call: funder memo
tests/test_synthetic.py # must return the $95k gap
```

Contract: `city_timelines.json` (agree by 11:45, one row per city)

```
{
  "city": "Santa Ana",
  "median_months_to_permit": null,
  "n_projects": 0,
  "tenure": "owner | all",
  "method": "APR join on APN | ASSUMPTION fallback",
  "source": "HCD APR Table A2 via data.ca.gov",
  "checked": "2026-11-07"
}
```

Contract: `programs.json` row

```
{
  "name": "CalHFA Dream For All",
  "max_assistance": "up to 20%, cap $150,000",
  "income_table": "state | home",
  "status": "confirmed | funds_unconfirmed | unknown",
  "source_url": "https://calhfa.ca.gov/dream/",
  "checked": "2026-11-07"
}
```

Null and zero values are placeholders, filled by the delay track from real data. Every engine output carries `value`, `source` and `label` (`SOURCED` or `ASSUMPTION`) so the sources panel can print it.

Data sources

Load every source once into a committed file and read from disk during the demo. Statuses come from the lane repo's DATA-SOURCES.md files (checked 2026-11-06).

Data	Use	How to load	File	Status
HCD 2026 State Income Limits (revised June 23, 2026)	Income bands for state programs	Type the Orange County row (1–8 persons) by hand, with page number	`data/income_limits.json`	VERIFIED
HCD 2026 State HOME Income Limits (effective June 1, 2026)	Caps for HOME-funded buyers	Same, separate table	`data/income_limits.json`	VERIFIED
Health and Safety Code 50052.5	Affordable owner cost formula	Read in a browser (blocks scripts), cite section	`engine/affordability.py`	VERIFIED
Freddie Mac PMMS history CSV	30-year rate; 7.18% on 2026-11-05 vs 7.40% on 2026-10-08	Download once, commit	`data/pmms.csv`	VERIFIED
HCD Annual Progress Report data (Table A2, via data.ca.gov)	Dates approved, permitted, completed per project	Download OC rows once; join across years	`data/city_timelines.json`	Fields confirmed in HCD instructions; join not tested
Lane 2 timeline.json	Fallback 4-year baseline	Copy	same	ASSUMPTION

Data	Use	How to load	File	Status
Program pages in Lane 3 DATA-SOURCES.md, section 4	Funding stack rules	Claude drafts each row from the page; a human checks	`data/programs.json`	Mixed: see below
Synthetic example deal	Test the math	Copy	`data/synthetic_deal.json`	SYNTHETIC, never shown as real

Programs to load first (5–6 rows): CalHFA MyHome (page: up to 3.5% FHA / 3% conventional), CalHFA Dream For All (page: up to 20%, cap $150,000, drawing; funds unconfirmed), CalHome via the HOSN NOFA, FHLBank SF WISH, Santa Ana My First Home (read terms on the page, not a snippet), OC Housing Finance Trust. Do not model Forgivable Equity Builder as open.

APR data gotcha: each date is filled only in the year it happened, so one project spans several rows across years. Join on Current APN or the Local Jurisdiction Tracking ID. Filter tenure to O (owner) where enough rows exist.

Features

Build in this order. Each must-have keeps the demo runnable on its own; stretch items only start once all must-haves work.

Must-have

- [ ] Project form: city, number of homes, cost per home by line, income mix (bands and household sizes)
- [ ] Affordable price per home from income limits + rate, table name and revision date shown
- [ ] Gap per home, reproducing the synthetic deal's $95k
- [ ] Time cost line from `city_timelines.json` (or the fallback slider), with project count per city
- [ ] Programs list with status badges: confirmed, funds unconfirmed, unknown
- [ ] Waterfall chart: cost → family price → each program → fundraising left
- [ ] Sources panel: every number links to its source or is tagged ASSUMPTION
- [ ] "Prototype, not financial or legal advice" banner

Should-have (the three repo scenarios)

- [ ] M01: which sources can I stack, gap recalculated
- [ ] M02: rate slider, gap change in dollars (use the 7.40% → 7.18% move)
- [ ] M03: donor view, "$25k covers these lines of this home"
- [ ] City A vs City B comparison: same project, gap difference from time
- [ ] Claude funder memo: one page, every figure passed in from the engine

Stretch

- [ ] Claude reads a new program PDF and drafts a `programs.json` row with citations, for a human to approve
- [ ] Stale-data warning when a table's revision date is older than a newer one on file

Team and timeline

Two tracks build in parallel and meet at 1:00 PM through one file, `city_timelines.json`. Fill in names.

Role	Owner	Builds
Delay track (2 people)		APR data → `city_timelines.json`, months to permit per city, project counts
Engine builder		Affordability, gap and stack math; tests against the synthetic deal
Data		`income_limits.json`, `pmms.csv`, `programs.json` with sources and dates
Design + UI		Form, waterfall chart, status badges, sources panel
Presenter		README from `templates/TEAM-README.md`, user story, video script, practitioner quote

Checkpoints

1. **11:45** — Agree on the `city_timelines.json` format (see Architecture). Repo created from the template; app skeleton runs.
2. **12:30** — Engine reproduces the synthetic $95k gap. **Delay track decision:** if the cross-year APR join is not working, switch to the fallback (timeline.json baseline as ASSUMPTION + a months slider; real data only for cities that joined).
3. **1:00** — Merge: time cost flows into the gap. Waterfall on screen.
4. **1:45** — M01, M02, M03 work end to end. Funder memo works. Sources panel complete.

5. **2:00** — Freeze features. README done. Repo public, no keys or personal information in history.
6. **2:20** — Video recorded and form submitted (10 minutes of buffer before 2:30).

Demo script

Aim for 30–45 seconds, one take, real screen. Fill the brackets with what the tool actually shows.

1. **(0–5s)** "A nonprofit wants to build 8 for-sale homes in Orange County. Each one is $[X] short."
2. **(5–15s)** Enter the project. The family price appears with its source: income table, revision date, today's rate.
3. **(15–22s)** Switch City A to City B. "Waiting on the city adds $[Y] per home here, based on [n] past projects."
4. **(22–32s)** The waterfall drops: programs fill the gap, one flagged "funds unconfirmed." $[Z] left to fundraise.
5. **(32–38s)** Move the rate slider from 7.40% to 7.18%. The gap moves by $[W].
6. **(38–45s)** Click a number to show its source. "Today this takes weeks of spreadsheets. This took 10 minutes."

For the live final, the same flow plus the donor view (M03) and the funder memo.

Rules and risks

Judges reward honesty: "Made-up numbers presented as facts" do not place. The repo's `CLAUDE.md` loads these rules into Claude Code automatically.

Rules

- Never invent a statute, number, fee or address. Unknown means "unknown" on screen.
- Tag every guessed number `ASSUMPTION` in code, UI and README, with what it is based on.
- Every number carries a source: URL, section and date checked.
- No personal information about real people. No API keys in git (use `.env`).
- Public or provided data only; download once, never hammer a city site.

Risks and fallbacks

Risk	Fallback
APR rows do not join across years	At 12:30, switch to timeline.json baseline + months slider; real data only where it joined
Too few owner (O) projects per city	Use all tenures and say so; show project count; "unknown" under a minimum count
Statute percentages misread	Show the section and quoted text beside the formula; a teammate double-checks
A program's funds have run out	Status badge, never counted as confirmed money
Live sites slow or blocked during demo	Everything served from committed files

What not to say

- "Nothing like this exists." Say instead: existing tools are rental-only (Urban Institute, Terner), buyer-level (Down Payment Resource) or static (Habitat's 2017 calculator).
- "Delay." Say "time to permit": some of it is normal processing.
- "A 10-month delay grows fundraising 7x" as a fact. It is from the synthetic example, labeled ASSUMPTION.

Submission checklist

Due 2:30 PM on the hub (`#submit`), one per team; judges look at the main branch and flag commits after 2:30. From docs/submission.md.

- [ ] Lane on the form: pick 3 Fund it (closest lane; we bent the brief to include Lane 2)
- [ ] Repo is public, checked in a private browser window; the form gets the repo's main page URL
- [ ] README follows `templates/TEAM-README.md`: real user, what we built, how Claude is used (and what a person checks), Try it, sources and assumptions, real vs mocked, where it gets things wrong, next steps, data and privacy, license
- [ ] "Try it" has exact commands for a fresh laptop, pinned versions, and the committed sample data
- [ ] `.env.example` with variable names only; no keys or personal information anywhere in history
- [ ] Every made-up number labeled ASSUMPTION
- [ ] Video (MP4/MOV, up to 2 min, 30 s ideal) uploaded, or a link that opens signed out

- [ ] Form fields: one-liner, user before/after, what it does, sources and assumptions, repo URL, video
- [ ] Someone outside the team could run it from the README